A Bit-Exact, Processor-Free FPGA Adaptive Noise Canceller: Fair Comparison of LMS, NLMS and Divider-Free NLMS-Shift on Cyclone IV E

Satvik Garg[1], Aditya Gupta[1], Jaffino G[1*]

[1*]School of Electronics Engineering (SENSE), Vellore Institute of Technology, Vellore, India.

*Corresponding author(s). E-mail(s): [TBD: e-mail; author order to be confirmed by the authors];

Abstract

Cancelling noise with a reference microphone is one of the oldest applications of the least-mean-square (LMS) family, and a small field-programmable gate array (FPGA) can do it at audio rates with no processor at all. We present ULLAP, a 12-bit, 32-tap noise canceller built for the DE2-115 board (Cyclone IV E) and its WM8731 codec. A single serial core, sharing one multiplier, runs three algorithms: LMS, normalised LMS (NLMS) with a restoring divider, and NLMS-Shift, in which a leading-one detector and a shift take the place of the division. With step sizes tuned on held-out seeds, NLMS-Shift stays within the 95% confidence interval of NLMS on stationary and amplitude-step noise (22.37 and 24.33 dB over 20 seeds), beats LMS by about 2 dB on amplitude steps, and needs 72 rather than 130 busy cycles per sample. We show that NLMS-Shift is exactly NLMS with a level-dependent step lying between 0.5 and 1 times the nominal value. On real recorded noise and speech the filters remove 11.7–14.9 dB and lift short-time objective intelligibility from 0.69–0.78 to 0.95–0.98; with the speech taken out they remove 18.5–31.9 dB, so it is the speech, not the noise, that sets the limit. A 12-bit word holds the quantisation loss under 0.4 dB on broadband noise. On the board, every one of the 724,459 captured output samples from the three algorithms matches the integer model, and the measured sample rate, latency and busy cycles agree with the register-transfer-level design.

Keywords: adaptive noise cancellation, NLMS, divider-free normalisation, FPGA, fixed-point arithmetic, bit-exact verification

1 Introduction

Most of us work in noise without noticing it any more: the hum of an office air conditioner, a compressor beside a production line, road noise inside a car, the chatter that surrounds someone wearing a hearing aid. If a second microphone can be placed near the noise source, the noise arriving at the main microphone can be predicted from it and subtracted. The predictor is an adaptive filter. It learns the acoustic path between the two microphones while it runs and follows that path when it changes. For this filter the least-mean-square (LMS) algorithm and its normalised form (NLMS) are still the default. They cost only a few multiply–accumulate operations per tap and sample, and their behaviour has been studied for decades [1].

A small embedded device has to run this filter in real time, with a short and fixed delay, on very little energy. A field-programmable gate array (FPGA) suits that job well. The datapath is built around the algorithm, the latency is known to the clock cycle, and there is no processor or operating system to schedule. In hardware, though, LMS and NLMS are not equally easy. LMS needs nothing beyond multipliers and adders, but how fast it converges depends on how loud the input is. NLMS removes that dependence by dividing the step size by the input power, which costs one division per sample. Dividers are slow and take a lot of logic, so recent designs approximate the division instead: with an approximate Newton–Raphson iteration [2], with canonical-signed-digit reciprocals [3], or with a binary reciprocal built from a leading-one detector and a shifter [4]. The crudest option rounds the input power to a power of two, and the division turns into a plain shift. We call this variant NLMS-Shift.

Two things kept bothering us as we read this literature. The first is tuning. Variants are rarely tuned with the same care, so a reported gain may come from a weak baseline rather than from the algorithm. We fell into this ourselves: an early version of this work showed NLMS-Shift 5.7 dB ahead of NLMS, and the gap vanished once NLMS was given a sensible regularisation constant. The second is the link between model and silicon. Hardware papers report area, clock rate and power. Those numbers show that a design fits on the chip, not that the chip produces the same samples as the model behind the accuracy results.

This paper describes ULLAP, a 12-bit, 32-tap adaptive noise canceller for the low-cost Terasic DE2-115 board (Intel Cyclone IV E) and its WM8731 audio codec, and uses it to address both concerns. Our main contributions are:

1. One serial core that runs LMS, NLMS (with a restoring divider) and NLMS-Shift on a single shared multiplier. The three algorithms therefore share the same datapath, word length and filter length. The core sits in a complete audio system without a processor, closes timing at 50 MHz on Cyclone IV E, and is released together with all scripts and data.
2. A fair comparison. Each algorithm's step size and regulariser are picked on held-out seeds by the same rule, and every result carries a 20-seed 95% confidence interval. The comparison covers synthetic scenarios and real recorded noise and speech, and includes a reproducible word-length study.
3. A bit-exact verification chain from a Python integer model through the RTL to the board wrapper (86 regression tests, codec, freeze and reset behaviour included). On

the board, captures of real codec samples show every output sample of all three algorithms equal to the model's.

4. Two explanations backed by measurement. The first concerns the shift. NLMS-Shift turns out to be NLMS with a step that moves between 0.5 and 1 times the nominal one as the input level changes. NLMS run with the average of that step reproduces NLMS-Shift within 0.3 dB on broadband noise, while on tonal input the step jumps whenever the window power crosses a power of two. The second concerns the real recordings. Once the speech is taken out, the same filters remove 18.5–31.9 dB of the same real noise. The lower noise reduction with speech is therefore caused by the speech disturbing the adaptation, not by the noise.

The shift normaliser itself is not new, and we do not claim it; Section 2 places it among earlier division-free designs.

Section 2 reviews the problem and the related work. Section 3 gives an overview of the system, Section 4 the algorithms and the fixed-point model, and Section 5 the hardware. Sections 6 and 7 cover verification and the experimental set-up, Section 8 the results, and Sections 9 and 10 the discussion and conclusions.

2 Literature review

2.1 The problem: noise in daily life

Noise is no longer treated as a mere nuisance; it is an environmental health risk. In Europe more than 113 million people live with transport noise above 55 dB(A), and traffic noise alone is estimated to cost about 1.3 million healthy life-years a year through its effect on the heart and blood vessels [5]. Hahad et al. [6] review evidence that long exposure to noise is also linked to mental-health disorders, and they discuss the biological routes that may explain it. Hearing is the other side of the problem. The World Health Organization puts the number of people living with some degree of hearing loss at about 1.5 billion, and expects it to reach 2.5 billion by 2050 [7]. For these listeners background noise is the main reason speech stays hard to follow, even with a hearing aid. Cutting noise at the microphone or near the ear is useful in many settings: beside roads, where urban traffic noise sits mostly below 2 kHz [8]; inside vehicle cabins [9, 10]; in ventilation ducts and air conditioners [11]; and in hearing aids [12–14]. Many of the devices involved are small, cheap and battery-powered, so the processing has to run in real time at low cost and low energy.

2.2 Applications of adaptive noise control

Yang et al. [8] recorded road traffic noise at five sites in Beijing and built a feedforward LMS prototype that removes close to 30 dB(A) of single- and dual-frequency noise below 2 kHz. Working in a car, Jiang et al. [9] made the FxLMS step depend on the reference norm and tuned the two parameters of that rule by particle-swarm optimisation. On a dSPACE board their system reached up to 12 dB below 100 Hz, and about 3 dB over 20–400 Hz in road tests, where plain FxLMS managed between −0.8 and 0.3 dB. Zhu et al. [10] drove a car's own loudspeakers to control the first four engine orders. They measured an average reduction of 6.2 dB at 52 ear-level

microphones, and in a jury test 93% of the listeners felt more comfortable with the system on. In hearing aids, Yin and Chen [12] weighted the error of a variable-step adaptive feedback canceller; on a real device it converged 35% faster than a PEM-IMLMS baseline and 60% faster than PEM-NLMS. Husstedt et al. [13] tested six commercial behind-the-ear devices and found that impulse noise reduction lowers C-weighted peak levels by up to 20 dB, or about 9 dB on average. A recent systematic review of speech enhancement for hearing aids [14] finds that deep networks perform best in complex scenes, while hybrids that keep an adaptive filter give a better balance of latency and computation on wearable devices.

2.3 Adaptive algorithms

Lu et al. surveyed ten years of active noise control algorithms in two parts, one on linear systems [1] and one on nonlinear systems [15]. In both, LMS-type updates remain the base on which most newer methods are built. A popular idea is to run two filters at once, one fast and one accurate, and combine them. Félix et al. [16] proposed a modified convex combination of FXLMS filters that converges faster and attenuates more than FXLMS and the earlier convex scheme. Zhu et al. [11] joined an adaptive FxLMS filter and a CNN–GRU network through a convex weight to cancel harmonic noise in a bent air-conditioner pipe. A different problem arises when the loudspeaker leaks back into the reference microphone and the loop turns unstable. Li et al. [17] handled it with a Youla-parameterised feedforward controller and an adaptive IIR update, tested on a duct. All of these methods cancel better, but they add operations, networks or whole extra filters to every sample. That is the opposite of what a small FPGA design needs.

2.4 Hardware realisations

On the hardware side, Thannoon and Hashim [18] combined a fast retimed RLS filter and a slow LMS filter on an FPGA to remove power-line interference from ECG records of the MIT-BIH database. They report 24.2% higher SNR and 50% faster convergence than conventional structures. Jagadeeswari et al. [19] applied cut-set retiming to delayed NLMS filters for audio on a Spartan-6 and cut the critical path from 29.96 to 13.39 ns. For underwater sonar, Karakaya and Aytar [20] built semi-parallel 16-bit NLMS and variable-step LMS filters on an Artix-7, raising the SNR from 8.85 dB to 29.36 and 28.57 dB with 2.51% and 2.91% of the LUTs. Park and Park [21] went a different way: a small CNN classifies the noise and selects one of several fixed filters, which in simulation gave 2.3 dB more SNR than LMS and 53% less dynamic power. Abdi et al. [22] built a narrowband controller around a variable-step LMS notch filter on an FPGA to suppress microphonics in a superconducting accelerator cavity. Other groups worked on the arithmetic itself. Nagabushanam et al. [23] built a multiplier-less LMS filter from distributed arithmetic with offset-binary coding, and da Rosa et al. [24] replaced the multipliers of an LMS power-line canceller with adders and shifters, saving up to 4.98 times in power and 30.46% in area.

2.5 Removing the division from NLMS

Closest to our work are the papers that attack the NLMS division directly. Guidotti et al. [2] saved power in an NLMS interference canceller with an approximate Newton–Raphson integer divider, and Coelho et al. [25] lowered the cost of adaptive filtering with sum-of-powers-of-two arithmetic. Yao and Huang [3] approximated the reciprocal of the input power with canonical-signed-digit encoded divisors in a 16-tap NLMS FIR filter. Huang and Yao [4] then proposed a binary reciprocal approximation for an NLMS IIR filter. On the same DE2-115 board we use, their design needs 5.07 nJ per sample against 8.33 nJ with a 15-bit SRT divider, and it lowers the energy per sample by 39.1%, 6.5%, 14.4% and 5.6% relative to SRT, Newton–Raphson, CSD and lookup-table methods [4]. They also point out that the older idea of rounding the divisor to a single power of two, which turns the division into a shift, has a large approximation error. That plain shift is exactly NLMS-Shift. Section 8.4 shows that its error acts only as a level-dependent step factor between 0.5 and 1.

We found no work that compares LMS, a true divider and a shift normaliser on one core, with equal tuning and statistical tests, and only a few that test a complete audio system with a codec on a board. ULLAP adds that equal-effort three-way comparison, a bit-exact chain from the model to the hardware, and an evaluation on real recordings.

3 System overview

Fig. 1 shows the whole system. Noise from the source v travels to the primary microphone through an unknown acoustic path h and mixes there with the wanted signal s. A reference microphone placed near the source picks up v itself. The two microphones feed the left and right LINE IN channels of the WM8731 codec, which digitises them and passes them to the FPGA over an I^{2}S link. In the FPGA, the I^{2}S receiver hands one pair $(x[n], d[n])$ per sample period to the ANC core. The core returns the error $e[n]$, which is the cleaned signal, and the I^{2}S transmitter sends it back to the codec DAC and LINE OUT. An I^{2}C engine programs the codec once after reset. Two slide switches select bypass and freeze the adaptation, a push button resets the system, and four GPIO test points bring out the sample strobe, the output strobe, the busy flag and the frame clock so that the timing can be measured. Everything is plain Verilog; there is no processor and no vendor IP.

4 Algorithms and fixed-point model

4.1 Signal model and cost function

The reference microphone sees the noise alone. The primary microphone sees the speech, the noise after the acoustic path, and a little sensor noise n_s:

$$x[n] = v[n], \tag{1}$$

$$d[n] = s[n] + \sum_{j=0}^{M-1} h_j \, v[n-j] + n_s[n]. \tag{2}$$

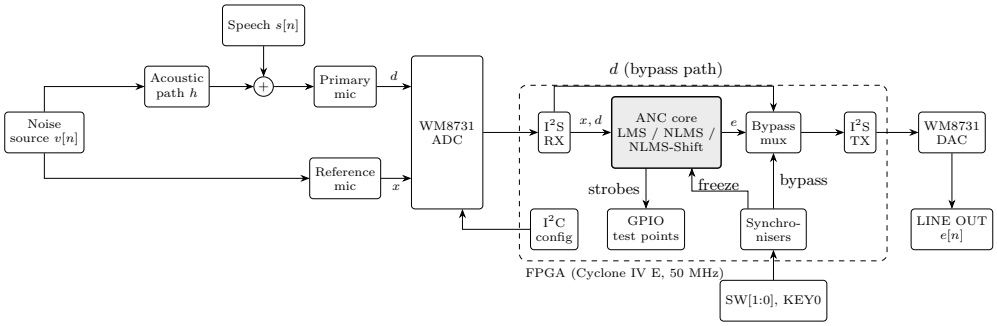


Fig. 1 Block diagram of the ULLAP system. The acoustic part on the left is simulated in the experiments (Section 7); everything from the codec on runs on the DE2-115 board.

Collecting the last L reference samples in $\mathbf{x}[n] = [x[n], \ldots, x[n-L+1]]^T$ and the weights in $\mathbf{w}[n] = [w_0[n], \ldots, w_{L-1}[n]]^T$, the filter estimates the noise part of $d[n]$, and what is left over is the system output:

$$y[n] = \mathbf{w}^T[n]\,\mathbf{x}[n] = \sum_{i=0}^{L-1} w_i[n]\,x[n-i], \tag{3}$$

$$e[n] = d[n] - y[n]. \tag{4}$$

Since s is uncorrelated with v, minimising the mean-square error

$$J(\mathbf{w}) = E\{e^2[n]\} = E\{d^2[n]\} - 2\,\mathbf{w}^T\mathbf{p} + \mathbf{w}^T\mathbf{R}\,\mathbf{w}, \tag{5}$$

$$\mathbf{R} = E\{\mathbf{x}[n]\,\mathbf{x}^T[n]\}, \qquad \mathbf{p} = E\{d[n]\,\mathbf{x}[n]\}, \tag{6}$$

removes the noise and leaves the speech alone. The minimum is the Wiener solution $\mathbf{w}_o = \mathbf{R}^{-1}\mathbf{p}$, which equals the acoustic path when $L \geq M$ and n_s is white.

4.2 LMS, NLMS and NLMS-Shift

LMS replaces the gradient $\nabla J = -2(\mathbf{p} - \mathbf{R}\mathbf{w})$ with its instantaneous estimate $-2e[n]\mathbf{x}[n]$. All three algorithms use the resulting update

$$\mathbf{w}[n+1] = \mathbf{w}[n] + \mu[n]\,e[n]\,\mathbf{x}[n]. \tag{7}$$

and differ only in the step $\mu[n]$. With the window power $P[n] = \|\mathbf{x}[n]\|^2$, the steps of LMS, NLMS and NLMS-Shift are

$$\mu_{\mathrm{LMS}}[n] = \mu, \qquad 0 < \mu < \frac{2}{L\,\sigma_x^2}, \tag{8}$$

$$\mu_{\mathrm{NLMS}}[n] = \frac{\tilde{\mu}}{P[n] + \epsilon}, \qquad 0 < \tilde{\mu} < 2, \tag{9}$$

$$\mu_{\mathrm{Shift}}[n] = 2^{-(k[n]+K)}, \tag{10}$$

$$k[n] = \lfloor \log_2(P[n] + \epsilon) \rfloor + 1, \tag{11}$$

where σ_x^2 is the reference variance and $\epsilon > 0$ is a small regulariser. The LMS bound moves with the input level, which is why a fixed μ ends up either too slow for quiet inputs or unstable for loud ones. The NLMS bound does not move. The hardware keeps the power up to date with the exact recursion

$$P[n] = P[n-1] + x^2[n] - x^2[n-L], \tag{12}$$

which needs two squarings per sample instead of L. In NLMS-Shift the exponent $k[n]$ is simply the bit length of $P[n] + \epsilon$. A leading-one detector finds it, and multiplying by $\mu[n]$ becomes an arithmetic right shift of $e[n]\,x[n-i]$ by $k[n] + K$ places.

Proposition 1. *For every n, the NLMS-Shift step (10) satisfies*

$$\frac{2^{-K}}{2\,(P[n] + \epsilon)} \;\le\; \mu_{Shift}[n] \;<\; \frac{2^{-K}}{P[n] + \epsilon}, \tag{13}$$

In other words, the NLMS-Shift step never falls more than a factor of two below the NLMS step with $\tilde{\mu} = 2^{-K}$, and it respects the NLMS stability bound $0 < \tilde{\mu} < 2$ for $K \ge 0$.

Proof. By (11), $2^{k[n]-1} \le P[n] + \epsilon < 2^{k[n]}$. Taking reciprocals and multiplying by 2^{-K} gives (13), since $\mu_{\text{Shift}}[n] = 2^{-K}\,2^{-k[n]}$. □

The exact relation is

$$\mu[n] = \frac{r[n]\,2^{-K}}{P[n] + \epsilon}, \qquad r[n] = \frac{P[n] + \epsilon}{2^{\lfloor \log_2(P[n]+\epsilon) \rfloor + 1}} \in [0.5, 1), \tag{14}$$

so NLMS-Shift is NLMS with the time-varying step $2^{-K} r[n]$ (Section 8.4). If the input level is unknown over several octaves, the fractional part $u = \log_2(P+\epsilon) - \lfloor \log_2(P+\epsilon) \rfloor$ is spread evenly over $[0, 1)$, and the mean ratio is

$$E\{r\} = \int_0^1 2^{u-1}\,du = \frac{1}{2\ln 2} \approx 0.72. \tag{15}$$

4.3 Fixed-point model

Signals and weights are signed $Q1.F$ numbers with $W = 12$ bits and $F = W - 1$ fraction bits. Using the rounding shift and the saturation

$$\text{rs}(a, m) = \left\lfloor \frac{a + 2^{m-1}}{2^m} \right\rfloor, \tag{16}$$

$$\text{sat}_B(a) = \min\!\big(\max\!\big(a, -2^{B-1}\big),\, 2^{B-1} - 1\big), \tag{17}$$

the integer datapath works on the integer samples $x_i = x[n-i] \cdot 2^F$ and weights w_i as

$$a[n] = \mathrm{sat}_{B_a}\Big(\sum_{i=0}^{L-1} w_i\, x_i\Big), \tag{18}$$

$$B_a = 2W + \lceil \log_2 L \rceil, \tag{19}$$

$$y[n] = \mathrm{sat}_W\big(\mathrm{rs}(a[n], F)\big), \tag{20}$$

$$e[n] = \mathrm{sat}_W\big(d[n] - y[n]\big), \tag{21}$$

$$w_i \leftarrow \mathrm{sat}_W\big(w_i + \Delta w_i\big), \tag{22}$$

with the weight increments

$$\Delta w_i^{\mathrm{LMS}} = \mathrm{rs}\big((M_\mu\, e[n])\, x_i,\ F + 10\big), \tag{23}$$

$$\Delta w_i^{\mathrm{NLMS}} = \mathrm{rs}\big(g[n]\, x_i,\ F\big), \tag{24}$$

$$g[n] = \mathrm{sat}_{W+4}\left(\left\lfloor \frac{2N[n] + Q[n]}{2Q[n]} \right\rfloor\right), \tag{25}$$

$$\Delta w_i^{\mathrm{Shift}} = \mathrm{rs}\big(e[n]\, x_i,\ b(Q[n]) - F + K\big), \tag{26}$$

for LMS, NLMS and NLMS-Shift. Here $\mu = M_\mu\, 2^{-10}$, $N[n] = M_\mu\, e[n]\, 2^{2F-10}$, $Q[n] = P[n] + 2^{2F-E}$ for $\epsilon = 2^{-E}$, and $b(\cdot)$ is the bit length of a positive integer. Every point where a value is narrowed saturates and sets a sticky flag, so an overflow can never go unnoticed. The integer model (`py/fixedpoint.py`, `py/rtl_model.py`) uses exactly these widths and produces the golden vectors.

5 Hardware architecture

5.1 Core

Fig. 2 shows the datapath of the core (`rtl/anc_core.v`, vendor-neutral Verilog). The core takes one sample pair per `in_valid` strobe and does all its arithmetic on one multiplier with a three-stage pipeline (operand registers, product register, consume stage). A finite-state machine walks through each sample. It spends two cycles squaring the new and the dropped reference sample for (12) (NLMS and NLMS-Shift only), L cycles on the filter sum (18) and one cycle on y and e (21). The step computation follows, and then L cycles of weight updates (22). NLMS adds a sequential restoring divider for the gain $g[n]$; NLMS-Shift has a leading-one detector and a shift-amount register in its place. A `freeze` input holds the weights while filtering goes on.

This schedule fixes both the input-to-output latency and the number of busy cycles per sample:

$$N_{\mathrm{lat}} = \begin{cases} L + 3 & \text{LMS,} \\ L + 5 & \text{NLMS, NLMS-Shift,} \end{cases} \tag{27}$$

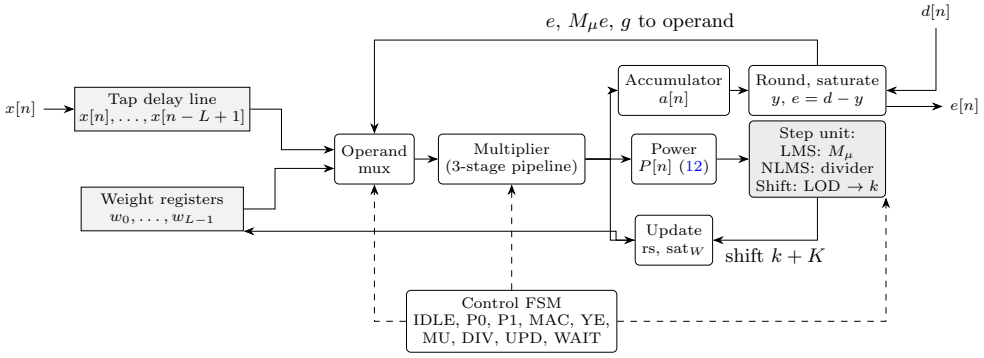


Fig. 2 Datapath of the ANC core. One multiplier is shared by the power update, the filter sum, the step computation and the weight updates; only the step unit differs between the three algorithms. Dashed lines are control signals.

$$N_{\text{busy}} = \begin{cases} 2L + 9 & \text{LMS}, \\ 2L + 13 + N_{\text{div}} & \text{NLMS}, \\ 2L + 8 & \text{NLMS-Shift}, \end{cases} \tag{28}$$

The restoring divider runs one iteration per dividend bit, $N_{\text{div}} = W + B_\mu + 2F + 4 + \lceil \log_2 L \rceil$, where B_μ is the signed width of M_μ, so the NLMS cycle count grows with the step-size constant. For $L = 32$ and the tuned settings the formulas give latencies of 35, 37 and 37 cycles and busy times of 73, 130 ($N_{\text{div}} = 53$) and 72 cycles, the same values we measured on the board (Section 8.6).

5.2 Board wrapper

Around the core, the DE2-115 wrapper holds an I^{2}C engine that sets up the WM8731 at reset (11 registers, each write acknowledge-checked), an I^{2}S receiver and transmitter, two-flop synchronisers on the reset and the switches, a frame-latched bypass multiplexer, an overrun flag, and four registered test points on the GPIO header. The I^{2}S blocks run on the 50 MHz system clock and sample the codec's bit and word clocks as data. The codec is the clock master, driven from a 12.5 MHz clock with its sample-rate register at 1536 fs, so

$$f_s = \frac{f_{\text{MCLK}}}{1536} = \frac{12.5\ \text{MHz}}{1536} = 8.138\ \text{kHz}, \tag{29}$$

$$N_s = \frac{f_{\text{clk}}}{f_s} = \frac{50\ \text{MHz}}{8.138\ \text{kHz}} = 6144, \tag{30}$$

which is the rate used in the simulations, with N_s clock cycles per sample. The core is busy for the fraction

$$U = \frac{N_{\text{busy}}}{N_s} \tag{31}$$

of each sample period. By (28) that is 1.19%, 2.12% and 1.17% for LMS, NLMS and NLMS-Shift. Again, no processor or vendor IP is involved.

6 Verification

Table 1 lists the verification steps. The core testbench checks every $y[n]$ and $e[n]$, the final weights, the sticky flags and the latency against golden vectors for 68 configurations (W from 8 to 31, L from 2 to 32, saturation corner cases, freeze patterns). The top-level testbench adds a behavioural WM8731 model that checks every I^2C write and follows the programmed sample rate. It runs tests T1–T10: configuration, NACK handling, bypass, end-to-end cancellation against golden vectors, freeze, reset, flags, clocks, a 45-point skew sweep, the 8 kHz rate and the GPIO timing. To make sure the testbenches can actually fail, we injected faults (mutation tests), and they caught them. Verilator `-Wall` lint reports no unwaived warning.

Table 1 Verification summary. RTL suites pass in both Icarus Verilog and Questa (`results/metrics/iverilog_regression_summary.txt`, `results/local_run_da0b6f4/`).

Suite	Tests	Result
Core golden vectors (`tb_anc_core`)	68	all pass, bit-exact
Protocol corner cases	4	pass
Freeze hold/toggle variants	3	pass
Top level with codec model (T1–T10)	11	pass
Python unit tests	53	pass
Verilator lint (`-Wall`)	–	0 unwaived
On-board Signal Tap captures (T9)	25	all bit-exact
On-board freeze, DAC word, 10-min soak	4	pass

7 Experimental set-up

The synthetic scenarios follow the reference model $x = v$, $d = s + h * v + n_s$ at 8 kHz with 16,000 samples. There are four: stationary coloured noise, amplitude steps ($\times 3$ then $\times 0.25$), an abrupt path switch at the midpoint, and a recorded chord as the noise source. We chose step sizes and regularisers on seeds 101–103, maximising the mean final noise reduction over the stationary, amplitude-step and path-switch scenarios with no overflow allowed. This gave LMS $\mu = 0.25$; NLMS $\mu = 0.25$, $\epsilon = 1$; and NLMS-Shift $K = 3$, $\epsilon = 0.25$. All reported results use seeds 1–20. The final noise reduction is taken over the last quarter $\mathcal{N}$ of the samples,

$$\mathrm{NR} = 10 \log_{10} \frac{\sum_{n \in \mathcal{N}} \big(d[n] - s[n]\big)^2}{\sum_{n \in \mathcal{N}} \big(e[n] - s[n]\big)^2} \ \mathrm{dB}, \tag{32}$$

and each table entry gives the mean over the $S = 20$ seeds with its 95% confidence interval,

$$\overline{\mathrm{NR}} \pm t_{0.975,\,S-1} \frac{\sigma_{\mathrm{NR}}}{\sqrt{S}}, \qquad t_{0.975,19} = 2.093. \tag{33}$$

To compare two algorithms A and B we use the paired per-seed differences $\Delta_k = \mathrm{NR}_{\mathrm{A},k} - \mathrm{NR}_{\mathrm{B},k}$. We call them equivalent when two one-sided tests (TOST) both reject, at $p < 0.05$, that the mean difference lies outside a margin $\pm\delta$ with $\delta = 0.5$ dB:

$$H_{01} : \ \bar{\Delta} \leq -\delta, \qquad H_{02} : \ \bar{\Delta} \geq \delta. \tag{34}$$

Use of generative AI. The authors used Claude (Anthropic) as an assistant throughout this work: to write and debug analysis scripts and testbench code, to search for and summarise literature, and to draft and edit the text of this manuscript. [TBD: Authors: add any other AI tools you used (for example for local runs or literature searches) and adjust the list above if needed.] The authors checked every reported number against the committed result files and board captures, checked each reference against its source, and take full responsibility for the content of this article.

For the real recordings we took four noise files (air conditioner, vacuum cleaner, copy machine, babble) and ten utterances from the public MS-SNSD corpus[1] and resampled them to 8 kHz. Each seed draws a noise segment, an utterance and a random 16-tap acoustic path, at 0 dB input SNR, with and without speech leaking into the reference at −20 dB. Noise reduction, the short-time objective intelligibility (STOI) measure and the narrow-band perceptual evaluation of speech quality (PESQ, ITU-T Recommendation P.862) are computed after a 2 s convergence window.

8 Results

8.1 Synthetic scenarios

Table 2 Final noise reduction, dB, mean [95% CI] over seeds 1–20, $W = 12$, $L = 32$ (`results/paper_numbers.csv`).

Scenario	LMS	NLMS	NLMS-Shift
Stationary	22.39 [22.12, 22.66]	22.28 [22.01, 22.54]	22.37 [22.11, 22.63]
Amplitude steps	22.36 [21.72, 23.00]	24.16 [23.58, 24.73]	24.33 [23.88, 24.78]
Path switch	18.95 [18.81, 19.09]	18.53 [18.39, 18.67]	18.91 [18.76, 19.06]
Recorded chord	5.69 [5.66, 5.73]	5.73 [5.69, 5.76]	5.46 [5.42, 5.50]

In Table 2, NLMS-Shift ties NLMS on stationary noise and amplitude steps, and ties LMS on stationary noise and the path switch. On amplitude steps both normalised algorithms beat LMS by about 2 dB. Here the fixed LMS step is either too slow after the drop or too noisy before it: no LMS step size reaches 22 dB on stationary noise and 24 dB on amplitude steps at the same time, while both NLMS variants do (Fig. 3). On the recorded chord NLMS-Shift loses 0.2–0.3 dB. The paired per-seed tests in Table 3 put a number on "tie". Every NLMS-Shift versus NLMS difference falls inside a ±0.5 dB equivalence margin (TOST, $p < 0.05$), even though some differences are

[1] https://github.com/microsoft/MS-SNSD

statistically non-zero, such as +0.38 dB on the path switch. The 1.8–2.0 dB advantage of the normalised algorithms over LMS on amplitude steps is significant ($p < 10^{-7}$).

Table 3 Paired differences in final NR over seeds 1–20 (`results/metrics/paired_stats.csv`).

Scenario	Pair	Mean diff. (dB)	95% CI	p (paired t)	TOST ±0.5 dB
Stationary	NLMS-Shift − NLMS	+0.10	[+0.01, +0.18]	0.028	equivalent
Stationary	NLMS-Shift − LMS	-0.02	[-0.07, +0.04]	0.58	equivalent
Stationary	NLMS − LMS	-0.11	[-0.18, -0.04]	0.0038	equivalent
Amplitude steps	NLMS-Shift − NLMS	+0.17	[-0.12, +0.47]	0.23	equivalent
Amplitude steps	NLMS-Shift − LMS	+1.97	[+1.51, +2.43]	2.8e-08	not equiv.
Amplitude steps	NLMS − LMS	+1.79	[+1.38, +2.21]	2.5e-08	not equiv.
Path switch	NLMS-Shift − NLMS	+0.38	[+0.30, +0.47]	1.9e-08	equivalent
Path switch	NLMS-Shift − LMS	-0.04	[-0.07, -0.01]	0.018	equivalent
Path switch	NLMS − LMS	-0.42	[-0.50, -0.34]	1.1e-09	equivalent
Recorded chord	NLMS-Shift − NLMS	-0.27	[-0.30, -0.24]	7.7e-14	equivalent
Recorded chord	NLMS-Shift − LMS	-0.24	[-0.26, -0.22]	2.6e-16	equivalent
Recorded chord	NLMS − LMS	+0.03	[+0.00, +0.06]	0.036	equivalent

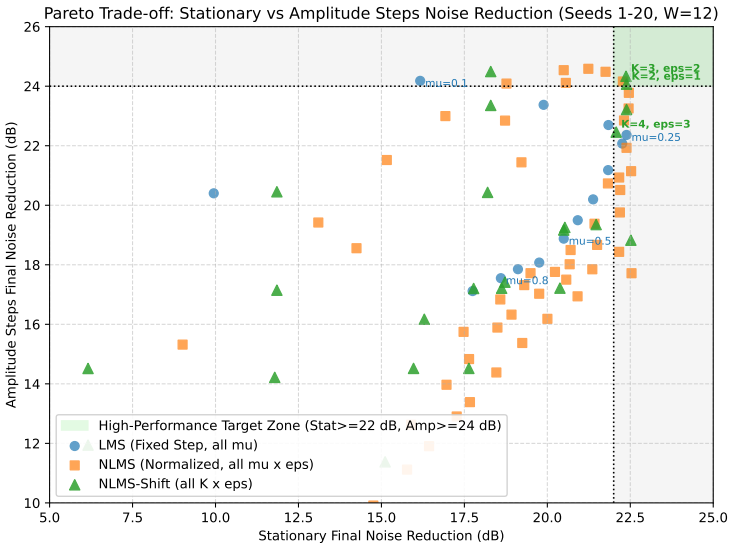


Fig. 3 Stationary versus amplitude-step noise reduction across step sizes (seeds 1–20).

Figures 4–6 show single runs (seed 1) at the fair-tuned settings: Fig. 4 the running noise reduction in each scenario, Fig. 5 the residual error power against the input noise power for stationary noise, and Fig. 6 the response to the amplitude steps. After the drop at 1.2 s, LMS settles about 3 dB below both normalised algorithms, which agrees with Table 2.

8.2 Real noise and speech

On all four real noises (Table 4) the filter roughly doubles PESQ and brings STOI close to 1. The ranking of the algorithms never changes: NLMS leads LMS by 0.4–0.6 dB,

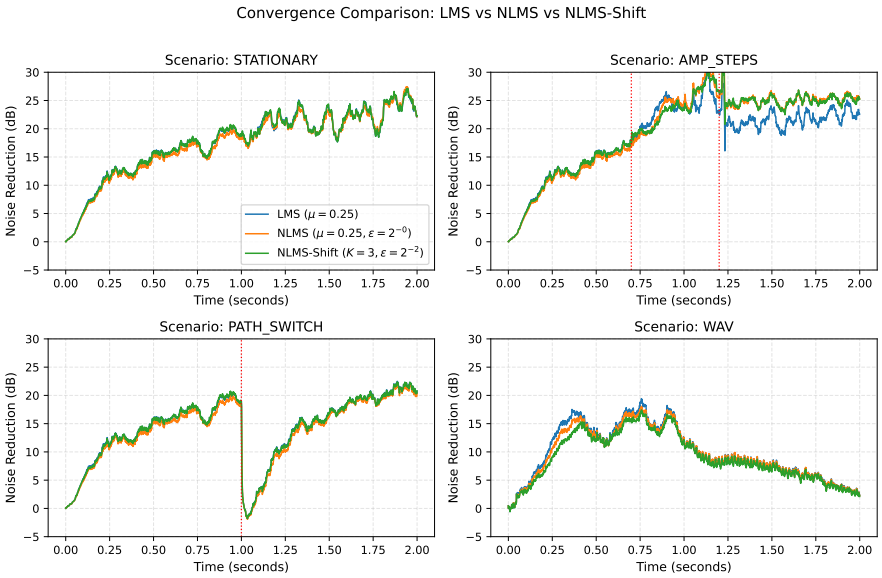


Fig. 4 Running noise reduction in the four synthetic scenarios (seed 1, $W = 12$, $L = 32$, fair-tuned settings). Dotted lines mark the amplitude steps and the path switch.

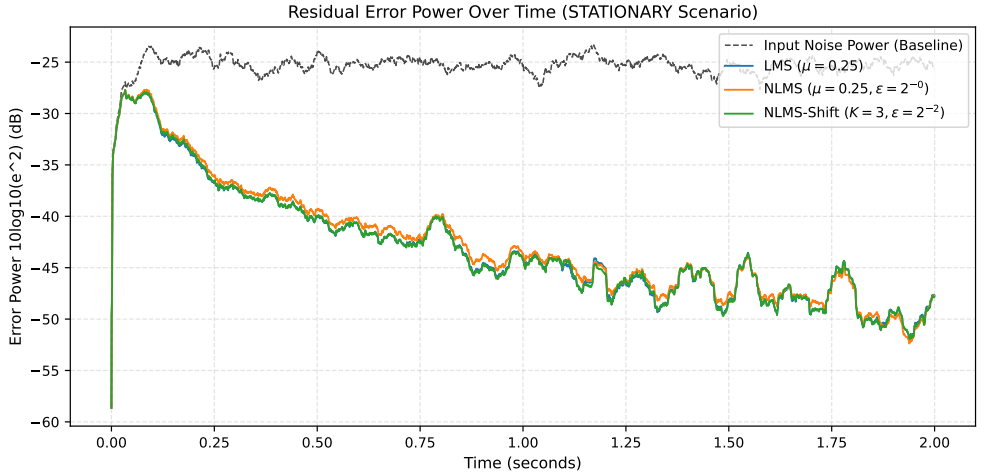


Fig. 5 Residual error power over time against the input noise power, stationary scenario (seed 1).

and NLMS-Shift trails NLMS by 0.15–0.35 dB. The noise reduction is lower than in Table 2, but the real noise is not harder to predict. When we removed the speech from the primary input, the same filters on the same noise segments and paths reached 18.5–19.2 dB on the air conditioner and 22.4–31.9 dB on the other three noises, as much as on synthetic stationary noise or more (`py/exp_real_audio_nospeech.py`, `results/metrics/real_audio_nospeech.csv`). The speech is uncorrelated with the reference,

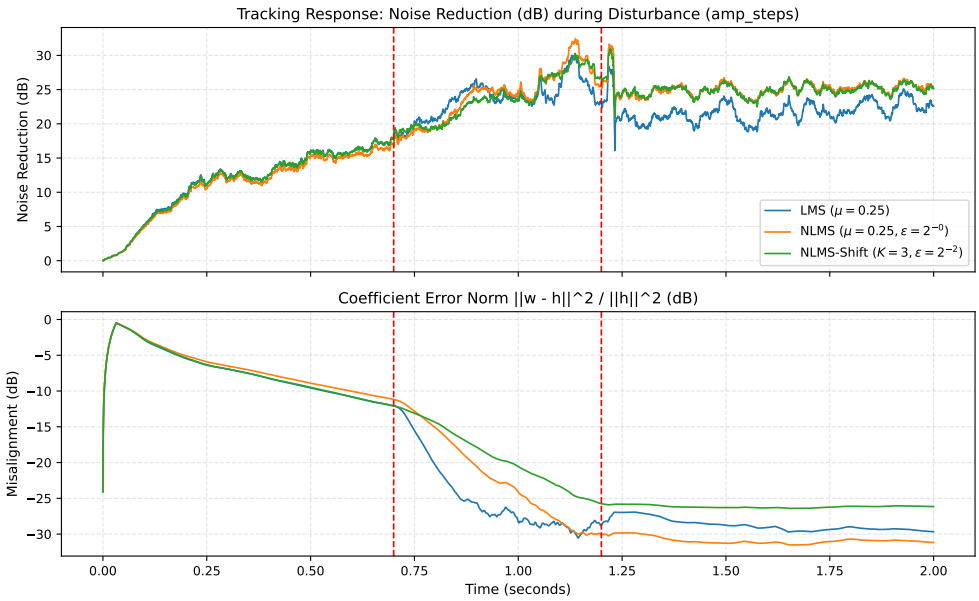


Fig. 6 Tracking of the amplitude-step disturbance (seed 1): noise reduction and coefficient misalignment.

Table 4 Real recorded noise and speech (MS-SNSD), fixed point $W = 12$, mean ± 95% CI over 20 seeds, no speech leakage into the reference; STOI and PESQ of e against s, with the unprocessed input in brackets (`results/metrics/real_audio_summary.csv`).

Noise	Metric	LMS	NLMS	NLMS-Shift
Air conditioner	NR (dB)	11.65 ± 0.75	12.04 ± 0.82	11.89 ± 0.80
	STOI (input 0.784)	0.948	0.947	0.948
	PESQ (input 1.54)	2.34	2.30	2.33
Vacuum cleaner	NR (dB)	14.32 ± 0.32	14.90 ± 0.33	14.56 ± 0.35
	STOI (input 0.730)	0.982	0.983	0.982
	PESQ (input 1.39)	2.81	2.81	2.81
Copy machine	NR (dB)	14.02 ± 0.38	14.63 ± 0.34	14.39 ± 0.32
	STOI (input 0.694)	0.977	0.978	0.977
	PESQ (input 1.36)	2.74	2.75	2.73
Babble	NR (dB)	11.95 ± 0.42	12.43 ± 0.49	12.27 ± 0.46
	STOI (input 0.719)	0.956	0.955	0.955
	PESQ (input 1.48)	2.48	2.47	2.47

but it still enters the error $e[n]$ that drives the weight update, and it costs 6.4–17.9 dB. Without speech LMS is ahead of both normalised variants on all four noises, and with speech it loses the most (7.6–17.9 dB against 6.4–14.8 dB for NLMS), so the order flips. The usual remedy is to slow down or stop adaptation while someone is speaking. The core's `freeze` input (Section 5.1) could be driven by a voice-activity detector for this,

but we have not evaluated that here. When −20 dB of the speech leaks into the reference, the filter also cancels part of the speech. Noise reduction drops by 2.3–4.6 dB and PESQ by 0.25–0.6, and saturation events appear (18,500 over 19.2 million processed samples, mostly for the air conditioner). Where the reference microphone sits therefore matters more than which algorithm is used (`paper/tab_real_audio_xt.tex`).

8.3 Word length

At the tuned settings, the worst-scenario loss against a floating-point run of the same algorithm is 4.1–5.9 dB at $W = 10$, 0.9–1.2 dB at $W = 11$ and 0.3–0.4 dB at $W = 12$ (Fig. 7). Eleven bits is the shortest word that stays within our 3 dB criterion; we kept 12 bits for margin. The tonal chord recording behaves oddly: there the fixed-point filter beats floating point by up to 3.4 dB at $W = 11$ and loses 1.3 dB at $W = 13$. On such a narrow-band signal quantisation changes the effective adaptation, so the chord result swings with the word length and should not be used to choose it.

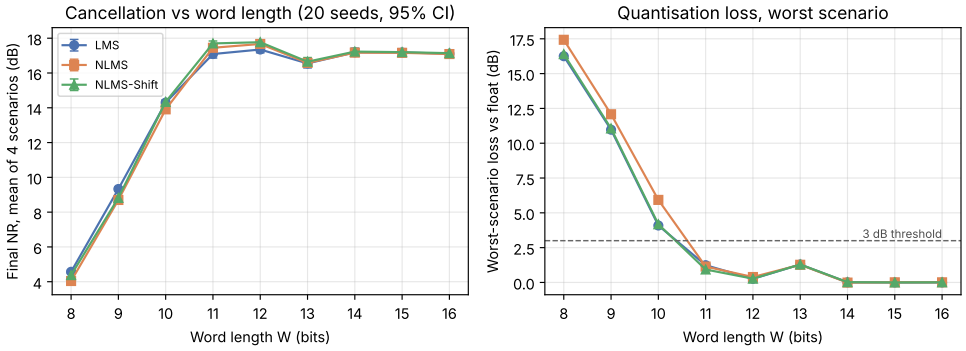


Fig. 7 Noise reduction and worst-scenario quantisation loss versus word length (20 seeds).

8.4 Equivalent step size of NLMS-Shift

By (14), NLMS-Shift and NLMS differ only in the factor $r[n]$. Averaged over input levels spread evenly on a logarithmic scale, $E[r] = \int_0^1 2^{u-1}\,du = 1/(2\ln 2) \approx 0.72$. At any one level, though, r depends on where $P + \epsilon$ sits inside its octave: it grows with the level and halves when $P + \epsilon$ crosses a power of two. With $L = 32$ and $\epsilon = 2^{-2}$, the crossing $P + \epsilon = 1/2$ falls at a reference RMS of −21.1 dBFS, 3 dB above the level used in the simulations. To see the effect, we swept the input level over 12 dB in 0.75 dB steps in the integer model (`py/exp_equiv_step.py`), on the stationary scenario (seeds 1–5) and on a two-tone reference (Fig. 8). On broadband noise the window power varies enough to smooth out the sawtooth, and the mean ratio $\bar{r}$ moves gradually between 0.56 and 0.78. On the tonal reference P is almost constant, and $\bar{r}$ drops from 0.87 to 0.62 within 0.75 dB of level. In both cases NLMS with the constant step $\bar{r}\,2^{-K}$ reproduces NLMS-Shift. The final noise reduction agrees within 0.27 dB

at all 17 broadband levels, and within 0.55 dB at every tonal level except the one that straddles the crossing (1.7 dB); the convergence time agrees within 8%. NLMS with the nominal step 2^{-K}, by contrast, is off by up to 3.5 dB and 48%.

The shift, then, changes nothing in the adaptation except the step size. It turns the fixed NLMS step into a step between 0.5 and 1 times the nominal one, set by the input level. On broadband input this is a smooth change of gain, and choosing K (and ϵ) for the expected level absorbs it. On tonal input the step changes by up to a factor of about 0.7 when the level crosses a power-of-two boundary of $P + \epsilon$, something a fixed-step NLMS never does.

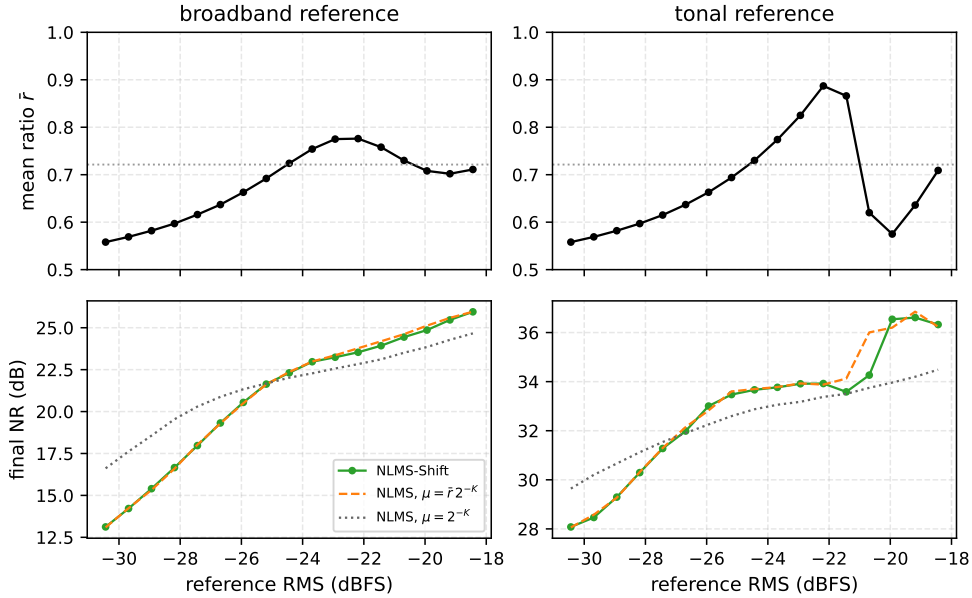


Fig. 8 Equivalent step of NLMS-Shift versus reference level (integer model, $W = 12$, $L = 32$, $K = 3$, $\epsilon = 2^{-2}$). Top: mean ratio $\bar{r}$ of (14); dotted: $1/(2 \ln 2)$. Bottom: final noise reduction of NLMS-Shift, of NLMS with step $\bar{r}\,2^{-K}$ and of NLMS with step 2^{-K}. Left: stationary scenario, seeds 1–5; right: two-tone reference (`results/metrics/equiv_step.csv`).

8.5 Hardware cost

LMS is about 31% smaller than either normalised variant. NLMS-Shift and NLMS need about the same logic (2,610 and 2,626 logic elements): the area saved on the divider goes to the leading-one detector, the shifter and the wider update path. NLMS-Shift does, however, have 15% fewer registers, needs 45% fewer busy cycles per sample and toggles 16% fewer bits per sample. All three designs meet the 50 MHz constraint with at least +2.953 ns setup slack and +0.401 ns hold slack, and without critical warnings. At 8.138 kHz even the busiest variant uses only 2% of the 6,144-cycle sample period, so one core could serve many channels or much longer filters.

Table 5 Complete board design (core, codec interface, synchronisers, status logic) on Cyclone IV E EP4CE115F29C7, $W = 12$, $L = 32$, Quartus Prime Lite 25.1, commit da0b6f4 (`results/quartus_reports/board_da0b6f4/`, `results/metrics/power_activity.csv`).

	LMS	NLMS	NLMS-Shift
Logic elements	1,792	2,626	2,610
Registers	1,123	1,404	1,186
9-bit multipliers	4	4	4
$F_{\max}$, slow 85 °C (MHz)	70.08	59.75	58.66
Setup slack at 50 MHz, slow 85 °C (ns)	5.731	3.263	2.953
Latency (cycles)	35	37	37
Busy cycles per sample (measured on the board)	73	130	72
RTL bit toggles per sample	5,668	7,146	6,012
Core dynamic power, vectorless (mW)	20.79	24.36	23.77

The power figures are Quartus vectorless estimates with the default 12.5% toggle rate, which Quartus itself rates as low confidence. Core static power (about 98.7 mW) dominates the 169 to 173 mW total. The dynamic estimates rank the designs LMS < NLMS-Shift < NLMS, in the same order as the toggle counts. A fixed toggle rate, however, mostly tracks area and ignores the fact that the datapath sits idle for 98% of each sample period, so these numbers are upper bounds, not energy per sample. We leave a gate-level, activity-annotated estimate for future work: the post-fit netlist flattens the hierarchy that the top-level testbench observes, so it needs a netlist-only testbench. A fair comparison with Huang and Yao [4], who use the same board, would report energy per sample as dynamic power divided by the sample rate from an activity-annotated Power Analyzer run. We do not have that yet and so claim no energy comparison with them. Their design is also fully parallel and runs at megasamples per second, while ULLAP shares one multiplier and runs at 8.138 kHz, so area and energy have to be compared per tap and per sample, not as raw totals.

8.6 On-board results

We recorded the board evidence with Signal Tap on the FPGA itself, so no external recorder or oscilloscope was needed. One logic-analyser instance stores, once per sample, the two codec samples entering the core (x and d, register outputs `x_reg` and `d_reg`) and the core output e. It uses a transitional storage qualifier on `out_valid`, triggers when the reset button is released, and holds 32,768 samples (4.0 s). A second instance samples the four GPIO test points at 50 MHz. The stimuli were the noise-only versions of the four scenarios, played from a laptop into LINE IN after a 1 kHz calibration tone. We pressed reset during the tone, so every capture starts from zero weights on real codec data at about −25 dBFS, and no sample in any capture reaches full scale. The analysis script and the raw exports are in the repository (`py/analyze_board_capture.py`, `board_evidence/`).

Bit-exactness. We took each of the 12 captures (three algorithms, four scenarios), started the integer model of Section 4 from zero state at the first sample after the reset, and fed it the captured x and d. It reproduced every captured $e[n]$: 351,723 samples

in total, 28,672 to 32,768 (3.5 to 4.0 s) per capture, without a single mismatch. The fixed-point arithmetic on the chip is therefore exactly the arithmetic we evaluated in simulation, even with analogue input carrying the codec's noise and offsets.

Timing. The GPIO capture shows an `in_valid` period of 6,144 clock cycles (8,138.02 Hz), an `in_valid` to `out_valid` spacing of 38, 36 and 38 cycles for NLMS-Shift, LMS and NLMS (the core latency plus one accept cycle, 0.76, 0.72 and 0.76 μs), and busy times of 72, 73 and 130 cycles. All of these equal the RTL simulation with the same parameters. The builds with Signal Tap still meet the 50 MHz constraint (setup slack +2.65, +5.68 and +3.78 ns); Table 5 refers to the builds without the logic analyser.

Fig. 10 shows the three algorithms on the same amplitude-step stimulus, aligned to the start of the scenario. (KEY0 was pressed by hand during the calibration tone, so the reset lands at a slightly different point of the stimulus in each capture.) All three settle to about 15 dB in the first 1.5 s. At 2.1 s the level of d rises by 7.4 dB, and they follow the step with only a short burst in e. The residual power after the step (-47 to -52 dBFS) is lower than before it (-44 dBFS), so the noise reduction climbs to 25 to 30 dB. Over the last quarter of the window the reduction is about 40 dB below 1.5 kHz and 11 to 15 dB near the band edge (3.5 to 3.9 kHz).

Noise reduction on the simulation signal. In a second session we played the 2 s, seed-1 noise-only signal of each scenario, i.e. exactly the 16,000-sample signal used in the simulations, once after the calibration tone, so that the whole scenario fits inside the capture. We located its start by correlating the captured x with the stimulus (correlation 0.95 to 0.99) and took $\mathrm{NR}_{\mathrm{final}}$ over the same last 25 % as in simulation. All 12 captures (28,672 samples after the reset each) are again bit-exact. Table 6 sets the board against the integer model on the same signal. The board comes out 2.9 to 10.6 dB below the simulation, by similar amounts for all three algorithms. The cause is the input level, not the hardware. The codec delivered x and d at about -27 dBFS, 3 dB below the simulation level, and at the tuned step sizes a lower input power slows adaptation within the 2 s. Run at the level the board actually received, the model matches the board within 0.2 dB on stationary and path-switch noise (Fig. 9). Amplitude steps and the recorded noise end with a quiet final quarter (d at about -41 and -57 dBFS), and there the board stays 1.7 to 8.1 dB below the level-matched model. At those levels two analogue effects are no longer small against d: the noise of the playback and codec path (-70 dBFS, measured in the silence after the stimulus) and an ADC offset that decays during the first second after the reset. For amplitude steps, even the best 32-tap least-squares filter fitted to the board samples of that window reaches only 24 dB, against 28 dB on the ideal signal. On the board, the three algorithms stay within 0.7 dB of one another in every scenario.

Freeze, DAC word and endurance. With NLMS-Shift running on stationary noise, we flipped SW[1] by hand 1.44 s into the scenario. The captured output equals the model with adaptation stopped from that sample on (28,672 samples, no mismatch). From the switch to the end of the scenario the noise reduction holds at 15.2 dB, against 15.4 dB just before the switch and 15.9 dB had adaptation continued. A third Signal Tap instance recorded the word sent to the DAC. In ANC mode it equals $\{e, 0000\}$, and in bypass (SW[0] = 1) its upper 12 bits equal d, in all 1,792 samples of each

Table 6 On-board NR_{final} (dB) on the 2 s seed-1 stimulus against the integer model on the same signal, at the simulation level and scaled to the level the board received (`results/board/session2/`). Every board capture is bit-exact.

Algorithm	Scenario	Board	Model, sim. level	Model, board level
LMS	stationary	15.88	22.77	15.76
	amplitude steps	17.13	27.71	25.23
	path switch	14.67	19.84	14.47
	recorded noise	5.99	9.11	7.78
NLMS	stationary	15.65	21.60	15.53
	amplitude steps	16.76	27.19	23.64
	path switch	14.03	19.06	14.04
	recorded noise	5.80	8.72	7.45
NLMS-Shift	stationary	15.80	22.68	15.78
	amplitude steps	16.57	25.79	22.10
	path switch	14.40	19.86	14.44
	recorded noise	5.46	8.63	7.32

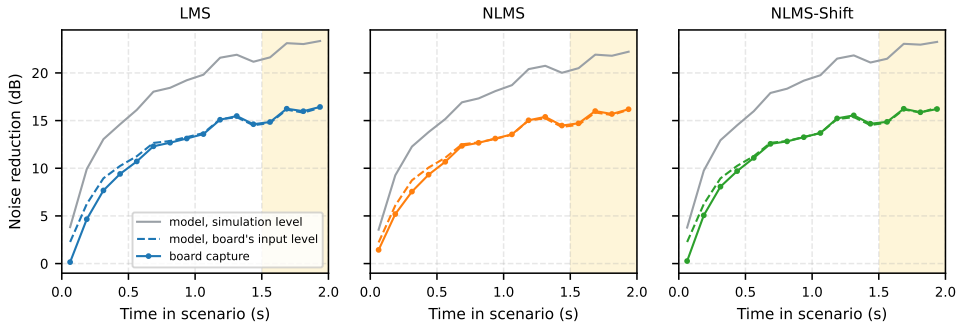


Fig. 9 Running noise reduction (0.125 s windows) on the 2 s stationary stimulus: board capture, integer model at the simulation level, and integer model with the stimulus scaled to the level the board received. Shaded: the NR_{final} window.

capture. We could not record the LINE OUT signal with the equipment we had, so this check takes the chain as far as the DAC interface. After 10 minutes of continuous operation on looped noise the GPIO timing had not changed (6,144 cycles per sample, 38 cycles from `in_valid` to `out_valid`, 72 busy cycles), and no saturation, NACK or overrun flag was lit. The build with all three logic-analyser instances still met timing (slack +2.93 ns).

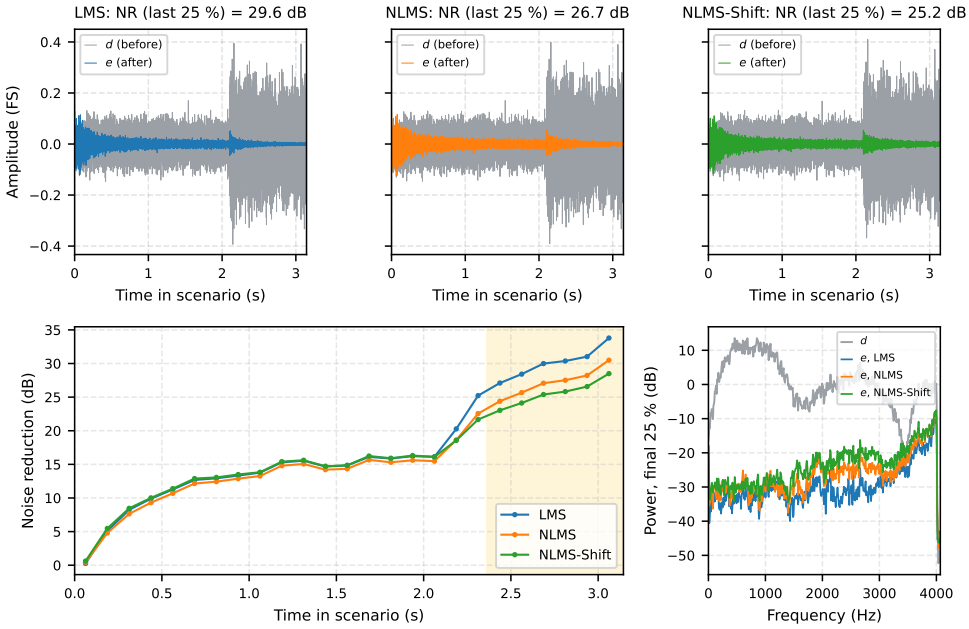


Fig. 10 On-board Signal Tap captures of LMS, NLMS and NLMS-Shift on the same amplitude-step noise, from the start of the scenario: primary input d and output e (top), running noise reduction in 0.125 s windows (bottom left; shaded: last 25 % of the common window) and power spectra over that window (bottom right). Every output sample of all three captures equals the integer model's.

9 Discussion and limitations

Our comparison supports a narrow conclusion. The divider-free normalisation keeps the level-independence of NLMS at roughly the cycle count of LMS. It is therefore the better choice when the noise level changes and throughput or energy per sample matters, while LMS remains the smallest design when the level is steady. It does not show that NLMS-Shift cancels better than NLMS, and Section 8.4 explains why. The shift only changes the step size, by a level-dependent factor between 0.5 and 1, so choosing K and ϵ for the expected input level matters more than the shift itself. Tonal noise near a power-of-two boundary of $P + \epsilon$ also sees a step jump that NLMS does not have. On real recordings, the speech at the primary input limits all three algorithms far more than the choice of algorithm does (6.4–17.9 dB, against at most 0.6 dB between algorithms).

The study has limits. The acoustic paths are simulated, also for the real recordings. Acoustic feedback (closed-loop howling) is not covered, and reference-based ANC is not the right structure for it anyway. Power is a switching-activity proxy until a gate-level analysis is done. On the board, the output is shown equal to the model up to the DAC word, but the analogue LINE OUT signal was not recorded. Finally, the absolute noise reduction depends on the input level. With the stimulus 3 dB below the simulation level, the board reached 14 to 17 dB instead of 19 to 28 dB on the

synthetic scenarios, and on stationary and path-switch noise the model reproduces this at that level. In a deployment, the codec input gain should therefore be set so that the reference uses most of the 12-bit range, or the step size should be retuned for the expected level.

10 Conclusion

ULLAP runs LMS, NLMS and NLMS-Shift on one serial 12-bit, 32-tap core on a low-cost Cyclone IV board, and compares them with equal tuning effort. NLMS-Shift matches NLMS within the 95% confidence interval on stationary and amplitude-step noise while needing 72 instead of 130 busy cycles per sample. All three remove 11.7 to 14.9 dB of real recorded noise, and 18.5 to 31.9 dB once the speech is taken out of the primary input, which shows that the speech, not the noise, sets the limit. NLMS-Shift is equivalent to NLMS with a step between 0.5 and 1 times the nominal one that depends on the input level. On the board, 25 Signal Tap captures of real codec data (724,459 samples) are bit-exact with the integer model, and so are the freeze behaviour and the DAC word. On stationary and path-switch noise, the measured noise reduction equals the model's, at the input level the board received, within 0.2 dB.

Declarations

Funding. [TBD: Authors: funding sources, or "No funding was received for this work."]
Competing interests. [TBD: Authors: competing interests, or "The authors declare no competing interests."]
Author contributions. [TBD: Authors: who did what (free text or CRediT).]
Data and code availability. All RTL, testbenches, scripts and result files are in the project repository [TBD: public URL and Zenodo DOI of the tagged release]. The real-audio study fetches its MS-SNSD files from a pinned commit and checks them by SHA-256.
Use of generative AI. Documented in the experimental set-up (Section 7).

References

[1] Lu, L., Yin, K.-L., Lamare, R.C., Zheng, Z., Yu, Y., Yang, X., Chen, B.: A survey on active noise control in the past decade–Part I: Linear systems. Signal Processing **183**, 108039 (2021) https://doi.org/10.1016/j.sigpro.2021.108039

[2] Guidotti, V., Paim, G., Rocha, L.M.G., Costa, E., Almeida, S., Bampi, S.: Power-efficient approximate Newton-Raphson integer divider applied to NLMS adaptive filter for high-quality interference cancelling. Circuits, Systems, and Signal Processing **39**, 5729–5757 (2020)

[3] Yao, C.-Y., Huang, Y.-Z.: The design of the NLMS adaptive filters using the fast-division approximation with CSD encoded divisors. IEEE Transactions on Circuits and Systems II: Express Briefs **71**(4), 2459–2463 (2024)

[4] Huang, Y.-Z., Yao, C.-Y.: Enhancing the power efficiency of the NLMS adaptive IIR filter with fast binary reciprocal approximation. Circuits, Systems, and Signal Processing **45**, 7561–7584 (2026) https://doi.org/10.1007/s00034-026-03536-z

[5] Münzel, T., Kuntic, M., Molitor, M., Sørensen, M., Daiber, A.: Transportation noise pollution as a cardiovascular risk factor: from epidemiological evidence to mechanistic insights. EXCLI Journal **24**, 1713–1739 (2025) https://doi.org/10.17179/excli2025-9050

[6] Hahad, O., Kuntic, M., Al-Kindi, S., Kuntic, I., Gilan, D., Petrowski, K., Daiber, A., Münzel, T.: Noise and mental health: evidence, mechanisms, and consequences. Journal of Exposure Science and Environmental Epidemiology **35**(1), 16–23 (2025) https://doi.org/10.1038/s41370-024-00642-5

[7] Chadha, S., Kamenov, K., Cieza, A.: The world report on hearing, 2021. Bulletin of the World Health Organization **99**(4), 242–242 (2021) https://doi.org/10.2471/BLT.21.285643

[8] Yang, B., Yin, J., Ye, Z., Yang, S., Wang, L.: Development and testing of an active noise control system for urban road traffic noise. Applied Sciences **14**(1), 175 (2024) https://doi.org/10.3390/app14010175

[9] Jiang, T., Liu, J., Peng, C., Wang, S.: Laboratory test of a vehicle active noise-control system based on an adaptive step size algorithm. Applied Sciences **13**(1), 225 (2023) https://doi.org/10.3390/app13010225

[10] Zhu, L., Zhong, X., Wang, H., Qiu, X., Wu, S., Mao, D.: An objective and subjective study on global active control of engine noise in a car. Journal of Vibration and Control **30**(9–10), 2311–2321 (2024) https://doi.org/10.1177/10775463231174730

[11] Zhu, W., Gu, Z., Chen, X., Xie, P., Luo, L., Bai, Z.: A new hybrid adaptive self-loading filter and GRU-net for active noise control in a right-angle bending pipe of an air conditioner. Sensors **25**(20), 6293 (2025) https://doi.org/10.3390/s25206293

[12] Yin, Y., Chen, F.: Acoustic feedback cancellation algorithm for hearing aids based on a weighted error adaptive filter. Electronics **12**(7), 1528 (2023) https://doi.org/10.3390/electronics12071528

[13] Husstedt, H., Hilgerdenaar, W., Frenz, M., Denk, F., Tchorz, J.: Evaluation of impulse noise reduction in hearing aids with technical measurements and ratings of discomfort. Acta Acustica **7**, 47 (2023) https://doi.org/10.1051/aacus/2023042

[14] Tetteh, P.D., Gyening, R.-M.O.M., Arthur, J., Owusu, J.A., Kponyo, B.A., Ahene, E., Kponyo, J.J.: Speech enhancement techniques for hearing aids: a systematic review. Universal Access in the Information Society **25**(3), 75 (2026)

https://doi.org/10.1007/s10209-026-01342-7

[15] Lu, L., Yin, K.-L., Lamare, R.C., Zheng, Z., Yu, Y., Yang, X., Chen, B.: A survey on active noise control in the past decade–Part II: Nonlinear systems. Signal Processing **181**, 107929 (2021) https://doi.org/10.1016/j.sigpro.2020.107929

[16] Félix, F.B., Castro Magalhães, M., Souza Papini, G.: Improved active noise control algorithm based on the convex combination method. Journal of the Brazilian Society of Mechanical Sciences and Engineering **43**, 163 (2021) https://doi.org/10.1007/s40430-021-02866-0

[17] Li, F., Ding, H., Wu, Z., Qian, F., Liu, T., Iqbal, A.: An improved stability compensation for feedforward active noise control systems with acoustic feedback. Journal of Mechanical Science and Technology **38**, 507–518 (2024) https://doi.org/10.1007/s12206-024-0101-5

[18] Thannoon, H.H., Hashim, I.A.: Hardware implementation of a high-speed adaptive filter using a combination of systolic and convex architectures. Circuits, Systems, and Signal Processing **43**, 1773–1791 (2024) https://doi.org/10.1007/s00034-023-02539-4

[19] Jagadeeswari, M., Rishabavarthani, P., Kumarganesh, S., Kannadhasan, S.: Enhanced adaptive filtering in audio systems: normalized least mean square with cut-set retiming. Analog Integrated Circuits and Signal Processing **126**(1), 12 (2026) https://doi.org/10.1007/s10470-025-02537-3

[20] Karakaya, E., Aytar, O.: FPGA implementation of NLMS and VSLMS algorithms for underwater acoustics applications. IEEE Access **14**, 53860–53872 (2026) https://doi.org/10.1109/ACCESS.2026.3681282

[21] Park, S., Park, D.: Low-power FPGA realization of lightweight active noise cancellation with CNN noise classification. Electronics **12**(11), 2511 (2023) https://doi.org/10.3390/electronics12112511

[22] Abdi, F., Cichalewski, W., Jałmużna, W., Butkowski, Ł., Branlard, J., Bellandi, A., Jabłoński, G.: FPGA-based implementation of an adaptive noise controller for continuous wave superconducting cavity. Electronics **13**(1), 155 (2024) https://doi.org/10.3390/electronics13010155

[23] Nagabushanam, M., Chakrasali, S., Gangadharaiah, S.L., Patel, S.H., Ramaiah, G., Basak, R.: An optimized VLSI implementation of the least mean square (LMS) adaptive filter architecture on the basis of distributed arithmetic approach. Journal of The Institution of Engineers (India): Series B **106**(3), 861–870 (2025) https://doi.org/10.1007/s40031-024-01028-9

[24] Rosa, M.M.A., Costa, P.Ü., Costa, E.A.C., Almeida, S.J.M., Paim, G., Bampi, S.: Design of a low power and robust VLSI power line interference canceler with

optimized arithmetic operators. Analog Integrated Circuits and Signal Processing **112**(2), 247–261 (2022) https://doi.org/10.1007/s10470-022-02050-x

[25] Coelho, L.F.d.S., Lovisolo, L., Tcheou, M.P.: Adaptive filtering with reduced computational complexity using SOPOT arithmetic. IEEE Transactions on Circuits and Systems I: Regular Papers **69**(2), 746–756 (2022)